ADVANCED RAG SERIES | PART 2 OF 5

Indexing, Chunking and Embeddings

Building the foundation of an Advanced RAG system

Covers: parsing, chunking strategies, parent-document retrieval, multi-vector indexing, RAPTOR, contextual retrieval, embeddings, HNSW and index maintenance.

A practitioner's guide: concepts, trade-offs, code and checklists

Contents

1. The Index Decides What Is Possible 4

1.1 The central tension: precision versus context . . . 4

2. Parsing: Getting Clean Text Out of Real Documents 4

2.1 What to look for . . . 4

2.2 Layout-aware and vision-based parsing . . . 5

3. Chunking Strategies 5

3.1 Fixed-size and recursive splitting . . . 5

3.2 Structure-aware splitting . . . 6

3.3 Semantic chunking . . . 6

3.4 Proposition indexing . . . 6

3.5 Chunk size is an empirical question . . . 6

4. Decoupling Search Units from Context Units 7

4.1 Parent-document (small-to-big) retrieval . . . 7

4.2 Sentence-window retrieval . . . 7

4.3 Multi-vector indexing: hypothetical questions and summaries . . . 7

4.4 Hierarchical summaries and RAPTOR . . . 8

5. Enrichment: Metadata and Contextual Chunks 8

5.1 Metadata . . . 8

5.2 Contextual retrieval . . . 8

5.3 Late chunking . . . 9

6. Embedding Models 9

6.1 How to choose . . . 9

6.2 Matryoshka and quantised embeddings . . . 9

6.3 Fine-tuning embeddings for your domain . . . 9

7. Vector Indexes and Approximate Search 10

7.1 The main index families . . . 10

7.2 HNSW parameters in practice . . . 10

7.3 Filtering and hybrid storage . . . 11

8. Keeping the Index Healthy 11

8.1 Incremental updates . . . 11

8.2 Deduplication and quality filters . . . 11

8.3 Security and access control at index time . . . 11

8.4 An indexing checklist . . . 11

9. End-to-End Example: Indexing a Product-Manual Corpus 12

9.1 The metadata schema . . . 12

9.2 The ingestion loop . . . 12

9.3 Querying with metadata filters 13

10. Designing Indexing Experiments 13

10.1 An experiment grid 14

10.2 Troubleshooting guide 14

10.3 Summary and further reading 14

1. The Index Decides What Is Possible

Retrieval can only return what the index contains, in the form the index stores it. If a crucial sentence was split in half during chunking, if a table was flattened into gibberish by the PDF parser, or if the embedding model has never seen your domain's vocabulary, no amount of clever querying or reranking will fully recover the loss. This is why experienced RAG engineers spend a disproportionate share of their time on the indexing pipeline: it is the foundation that every later stage inherits.

An advanced index is built through a sequence of deliberate decisions: how to parse sources into clean text and structure, how to cut that text into units, what to attach to each unit (metadata, summaries, context), which embedding model turns units into vectors, which approximate nearest-neighbour (ANN) structure stores the vectors, and how the whole thing is updated as documents change. This document walks through each decision, explains the trade-offs, and shows working code.

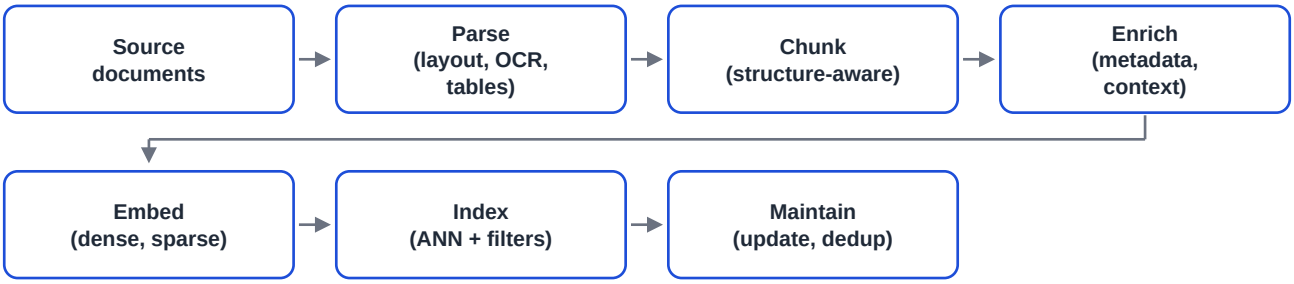


Figure 1. The indexing pipeline. Each stage can be tuned and evaluated independently.

1.1 The central tension: precision versus context

Every chunking decision balances two opposing forces. **Small chunks** produce focused embeddings: a chunk about one idea sits close to queries about that idea, so retrieval is precise. But a small chunk may lack the surrounding information the language model needs to answer, and pronouns or references inside it become ambiguous. **Large chunks** carry more context but dilute the embedding, because one vector must represent many ideas, and they consume more of the prompt budget. Nearly every technique in this document is a way to escape this trade-off: retrieve on small units, but hand the model large ones; or enrich small units with the context they have lost.

> **KEY IDEA** Decouple what you search over from what you give to the model. The best retrieval unit and the best generation unit are usually different sizes.

2. Parsing: Getting Clean Text Out of Real Documents

Before chunking, the document has to become text, and real documents are hostile. PDFs encode glyph positions rather than paragraphs; two-column layouts interleave when read naively; headers, footers and page numbers repeat on every page; tables become streams of numbers with no structure; scanned documents contain no text at all. The quality of your parsed text sets a ceiling on everything downstream.

2.1 What to look for

- **Reading order.** Multi-column layouts should be read column by column. Test with a representative sample and read the extracted text yourself.
- **Boilerplate removal.** Strip repeating headers, footers, navigation menus and cookie banners. They create near-duplicate chunks that crowd out real content.

- **Tables.** Extract tables as structured text (Markdown or CSV-like) and keep the caption and header row with the data. A table chunk without its header row is meaningless.
- **Figures and charts.** Either caption them with a vision model or store the surrounding text, and link the chunk to the image for display.
- **Scanned pages.** Use OCR, and store a confidence score so that low-quality pages can be flagged for review.
- **Structure.** Preserve headings, list levels and section numbers; they power structure-aware chunking and make excellent metadata.

2.2 Layout-aware and vision-based parsing

Modern parsers detect layout regions (title, paragraph, table, figure, caption) with a vision model and emit structured elements. For visually rich documents such as slide decks and infographics, an alternative is to skip text extraction and embed page images directly with a vision-language retriever. **ColPali** (Faysse et al., 2024) does this with a late-interaction approach over image patches, and showed strong results on visually complex document retrieval. It trades higher storage and compute for robustness against parsing errors.

> **TIP** Make parsing a separate, cached stage that writes structured intermediate output (JSON with element types and page numbers). Re-chunking and re-embedding then become cheap experiments instead of requiring the whole corpus to be re-parsed.

3. Chunking Strategies

Chunking converts a long document into retrievable units. The table below compares the main families. We then examine each in turn.

Strategy	How it splits	Strengths	Weaknesses
Fixed-size	Every N tokens, with overlap	Trivial, predictable	Cuts mid-sentence and mid-idea
Recursive character	Tries paragraph, then sentence, then word separators	Good default, respects natural breaks	Ignores document structure and meaning
Structure-aware	By headings, sections, code blocks, table rows	Chunks align with author intent; headings become metadata	Needs reliable parsing; uneven sizes
Semantic	Where embedding similarity between sentences drops	Topic-coherent chunks	Extra embedding cost; threshold tuning
Proposition-based	Atomic, self-contained factual statements	Very precise retrieval	Costly to generate; loses narrative flow
Agentic / LLM	LLM decides boundaries and may add summaries	Highest coherence	Slowest and most expensive

Table 1. Chunking strategies at a glance.

3.1 Fixed-size and recursive splitting

The recursive splitter is the workhorse default. It tries to split on paragraph breaks first; if a piece is still too long, it falls back to line breaks, then sentences, then words. This keeps related text together whenever possible. Typical starting points are 300 to 800 tokens with 10 to 20 percent overlap, but treat these as hypotheses to test, not truths. Overlap protects against an idea straddling a boundary, at the price of storing and retrieving duplicated text.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter.from_tiktoken_encoder(
    encoding_name="cl100k_base",
    chunk_size=500,          # tokens, not characters
    chunk_overlap=80,
    separators=["\n\n", "\n", ". ", " ", ""],
)
chunks = splitter.split_documents(docs)
print(len(chunks), "chunks; median tokens ~", 500)
```

Listing 1. Token-aware recursive splitting.

3.2 Structure-aware splitting

When documents have reliable structure (Markdown, HTML, reStructuredText, source code, legal articles), split on that structure instead of on character counts. A header-based splitter yields one chunk per section and records the heading path, for example "Warranty > Exclusions > Water damage", as metadata. That path is gold: prepend it to the chunk text before embedding so the vector carries the section context, and use it for filters and citations.

```
from langchain_text_splitters import MarkdownHeaderTextSplitter

md_splitter = MarkdownHeaderTextSplitter(
    headers_to_split_on=[("#", "h1"), ("##", "h2"), ("###", "h3")],
    strip_headers=False,
)
sections = md_splitter.split_text(markdown_text)

for sec in sections:
    path = " > ".join(v for k, v in sorted(sec.metadata.items()))
    sec.page_content = f"{path}\n\n{sec.page_content}"   # heading path travels with text
```

Listing 2. Header-based splitting; the heading path is prepended to the chunk.

3.3 Semantic chunking

Semantic chunking embeds each sentence (or small window of sentences), measures the distance between consecutive embeddings, and inserts a boundary where the distance spikes, signalling a topic shift. The result is variable-length chunks that each cover a coherent topic. Its benefit over recursive splitting is real on long, flowing prose, but modest on well-structured documents, and it multiplies embedding calls during ingestion. Compare it against a strong recursive baseline on your own questions before committing.

3.4 Proposition indexing

Dense X Retrieval (Chen et al., 2023) proposed indexing **propositions**: atomic, self-contained statements, rewritten so that they make sense in isolation (pronouns replaced by the entities they refer to). "It was founded in 1998" becomes "Google was founded in 1998". A query about the founding year then matches one crisp statement rather than a long paragraph. Propositions are generated with an LLM during ingestion, so the approach is expensive, and it works best for fact-dense corpora such as encyclopaedic and reference content.

3.5 Chunk size is an empirical question

There is no universally optimal chunk size. Factoid questions favour small chunks; explanatory and summarising questions favour large ones. The right procedure is to build a small evaluation set (Part 5), index the corpus at two or three sizes, and compare retrieval recall and end-to-end answer quality. Many teams find that the winning approach is not a single size at all but the hierarchical strategies in the next section.

4. Decoupling Search Units from Context Units

4.1 Parent-document (small-to-big) retrieval

In parent-document retrieval you split each document twice: into large **parent** chunks (say 1,500 tokens) and, within each parent, small **child** chunks (say 250 tokens). Only the children are embedded and searched. When a child matches, the system returns its **parent** to the language model. Search is precise because it runs on small, focused vectors; generation is well informed because the model sees the surrounding context.

```
from langchain.retrievers import ParentDocumentRetriever
from langchain.storage import InMemoryStore
from langchain_chroma import Chroma

parent_splitter = RecursiveCharacterTextSplitter(chunk_size=1500, chunk_overlap=150)
child_splitter  = RecursiveCharacterTextSplitter(chunk_size=250,  chunk_overlap=30)

retriever = ParentDocumentRetriever(
    vectorstore=Chroma(collection_name="children", embedding_function=embeddings),
    docstore=InMemoryStore(),          # holds the parents; use Redis/Postgres in prod
    child_splitter=child_splitter,
    parent_splitter=parent_splitter,
)
retriever.add_documents(docs)
parents = retriever.invoke("What are the warranty exclusions?")
```

Listing 3. Parent-document retrieval: search small, return big.

4.2 Sentence-window retrieval

Sentence-window retrieval is a finer-grained variant. Every sentence is embedded individually, which gives very sharp matching. At query time, the retrieved sentence is expanded by a window of neighbouring sentences (for example, two before and two after) before being passed to the model. The window size becomes a tunable parameter at query time, without re-indexing.

4.3 Multi-vector indexing: hypothetical questions and summaries

Instead of embedding the raw chunk, you can embed **something else that points to it**: a summary, or a list of questions the chunk could answer. The retriever searches these derived texts and returns the original chunk. Question-style indexing is particularly effective because user queries are questions; the embedding of "How do I reset my password?" is much closer to a stored question than to a paragraph of account documentation. A single chunk can hold several vectors (the raw text, a summary, three questions), all mapping to the same chunk identifier.

```
import uuid
from langchain_core.documents import Document

question_chain = (ChatPromptTemplate.from_template(
    "Write 3 distinct questions that the following passage answers. "
    "One per line, no numbering.\n\n{text}") | llm | StrOutputParser())

index_docs, store = [], {}
for chunk in chunks:
    cid = str(uuid.uuid4())
    store[cid] = chunk                                  # the original chunk
    for q in question_chain.invoke({"text": chunk.page_content}).splitlines():
        if q.strip():
            index_docs.append(Document(page_content=q.strip(), metadata={"chunk_id": cid}))

vectorstore.add_documents(index_docs)                   # search over questions
hits = vectorstore.similarity_search(user_query, k=5)
results = [store[h.metadata["chunk_id"]] for h in hits]   # return the originals
```

Listing 4. Hypothetical-question indexing: embed questions, return source chunks.

4.4 Hierarchical summaries and RAPTOR

Some questions are about the whole document or collection ("What are the main themes of this report?"), and no single chunk contains the answer. **RAPTOR** (Sarthi et al., 2024) builds a tree: leaf chunks are clustered by embedding similarity, each cluster is summarised by an LLM, and the summaries are themselves clustered and summarised, recursively, up to a root. All nodes at all levels are indexed together. A broad question retrieves high-level summary nodes; a detailed question retrieves leaves. This gives the system a way to answer at multiple levels of abstraction. The cost is a one-time summarisation pass, and the need to rebuild affected branches when documents change.

> **PITFALL** Derived indexes (questions, summaries, trees) are generated by an LLM and can contain errors or drift. Keep a reference from every derived vector back to its source chunk, never return the derived text as evidence, and spot-check samples regularly.

5. Enrichment: Metadata and Contextual Chunks

5.1 Metadata

Metadata turns a pile of vectors into a queryable knowledge base. Useful fields include the source document identifier, title, section path, page number, author, publication and modification dates, document type, language, product or version, access-control tags, and a content hash. Metadata enables exact filtering ("only documents from 2024 or later"), supports citations, powers self-query retrieval (Part 1), allows tenant isolation, and makes deletion and updating tractable. Capture it at ingestion; retrofitting it later is painful.

5.2 Contextual retrieval

A chunk lifted out of its document often loses the context that made it understandable. A chunk that says "Revenue grew by 3% over the previous quarter" does not say which company or which quarter. **Contextual Retrieval**, described by Anthropic in 2024, addresses this by asking an LLM to write a short context blurb (typically one or two sentences) for each chunk, situating it within the full document, and prepending that blurb to the chunk before embedding and before building the keyword index. Anthropic reported that contextual embeddings combined with contextual BM25 reduced failed top-20 retrievals substantially, and reranking reduced them further; the exact numbers depend on the dataset, so verify on yours.

```
ctx_prompt = ChatPromptTemplate.from_template(
    "<document>\n{doc}\n</document>\n"
    "Here is a chunk from the document:\n<chunk>\n{chunk}\n</chunk>\n"
    "Write 1-2 sentences situating this chunk within the overall document "
    "to improve search retrieval. Answer with only the context.")
ctx_chain = ctx_prompt | llm | StrOutputParser()

def contextualize(doc_text, chunk_text):
    ctx = ctx_chain.invoke({"doc": doc_text, "chunk": chunk_text})
    return f"{ctx}\n\n{chunk_text}"      # embed AND BM25-index this combined text
```

Listing 5. Contextual chunk enrichment (prompt caching makes this affordable for long documents).

The expensive part is passing the full document with every chunk. Prompt caching, available from several providers, makes this affordable because the document prefix is identical across the chunks of one document.

5.3 Late chunking

Late chunking (Jina AI, 2024) takes a different route to the same problem. A long-context embedding model processes the **whole document** (or a large window) in one pass, producing a contextualised vector for every token. Chunk vectors are then obtained by pooling the token vectors that fall inside each chunk boundary. Because each token's representation was computed with attention over the whole document, the chunk vector already "knows" the surrounding context. It needs no LLM calls, but requires an embedding model with a long context window and token-level outputs.

6. Embedding Models

The embedding model defines the geometry of your index: which texts count as "close". Choosing and, when needed, adapting it is one of the highest-leverage decisions in a RAG system.

6.1 How to choose

- **Benchmarks as a starting point.** The MTEB leaderboard compares models across retrieval, classification and clustering tasks. Use the retrieval subset and, if relevant, the language-specific tracks. Treat the score as a shortlist filter; leaderboard data may not reflect your domain.
- **Domain match.** Legal, biomedical, code and multilingual corpora each have specialised models. Test at least three candidates on your own labelled questions.
- **Dimensionality.** More dimensions can capture more nuance but cost memory and latency in proportion. 384 to 1536 dimensions are typical.
- **Context length.** The model's maximum input tokens must exceed your largest chunk. Models degrade near their limit, so leave headroom.
- **Asymmetric prefixes.** Many retrievers expect different instruction prefixes for queries and passages (for example "query: " versus "passage: "). Forgetting them silently hurts quality.
- **Cost and hosting.** API-based models are simple but add latency and per-token cost; local models give control and privacy. Changing models later requires re-embedding the entire corpus.

6.2 Matryoshka and quantised embeddings

Matryoshka representation learning (Kusupati et al., 2022) trains embeddings so that the first d dimensions form a useful vector in themselves. You can truncate a 1024-dimensional embedding to 256 dimensions with a modest loss in quality and a four-fold saving in storage and search time. A common pattern is a coarse first pass on short vectors followed by re-scoring the shortlist with the full vectors. Separately, **quantisation** stores each value in 8 bits or even 1 bit, shrinking indexes by 4 to 32 times; binary quantisation with a rescoring step keeps most of the accuracy for suitable models.

6.3 Fine-tuning embeddings for your domain

When generic models underperform on specialised vocabulary, fine-tune. The recipe uses triples of (query, relevant passage, hard negative). Queries can be written by domain experts or synthesised by an LLM from your passages; hard negatives are passages that look relevant but are not, often mined with the baseline retriever. A contrastive loss pulls queries toward their positives and pushes them from negatives. A few thousand good pairs often suffice for a noticeable gain, and adapters or small linear transformations offer a lighter alternative to full fine-tuning.

```
from sentence_transformers import SentenceTransformer, losses, InputExample
from torch.utils.data import DataLoader

model = SentenceTransformer("BAAI/bge-small-en-v1.5")

examples = [InputExample(texts=[q, pos, neg]) for q, pos, neg in triples]
loader = DataLoader(examples, shuffle=True, batch_size=32)
loss = losses.MultipleNegativesRankingLoss(model)   # in-batch negatives + the hard one

model.fit(train_objectives=[(loader, loss)], epochs=2, warmup_steps=100,
          output_path="bge-small-domain")
```

Listing 6. Contrastive fine-tuning of a bi-encoder (API details vary by library version).

> **NOTE** Fine-tuning should be justified by a measured gap. Always hold out an evaluation set, compare against the untuned model, and check that general-purpose behaviour has not collapsed.

7. Vector Indexes and Approximate Search

Exact nearest-neighbour search compares the query to every vector, which is fine for tens of thousands of vectors but too slow for millions. Production systems use **approximate nearest-neighbour (ANN)** indexes that trade a small, tunable loss in recall for large gains in speed.

7.1 The main index families

Index	Idea	Strengths	Trade-offs
Flat (brute force)	Compare against every vector	Exact; no tuning	Linear cost; small corpora only
IVF	Cluster vectors; search only the nearest clusters	Fast; moderate memory	Needs training; recall depends on nprobe
HNSW	Layered proximity graph; greedy navigation	Excellent recall and speed	High memory; slower inserts
PQ (product quantisation)	Compress vectors into short codes	Very small memory	Lossy; often combined with IVF
DiskANN-style	Graph index designed for SSD	Billion-scale on one machine	More complex deployment

Table 2. Common ANN index types.

7.2 HNSW parameters in practice

HNSW (Malkov and Yashunin, 2016) is the default in most vector databases. Three parameters matter. **M** is the number of links per node: higher values improve recall and raise memory use. **efConstruction** is the breadth of the search while building the graph: higher values produce a better graph at the cost of slower ingestion. **efSearch** is the breadth of the search at query time: it directly trades latency for recall and can be changed without rebuilding. Tune efSearch first, against a recall target (for example, 95 to 99 percent of what exact search would return) measured on real queries.

7.3 Filtering and hybrid storage

Metadata filters interact with ANN structures in subtle ways. **Post-filtering** retrieves the top k and then removes non-matching items, which can return fewer than k results or none. **Pre-filtering** restricts the candidate set first but may disrupt graph navigation. Modern databases implement filtered traversal that applies the predicate during the search. Check how your database handles highly selective filters (for example, a single tenant out of thousands) and test recall under them, not only without them. Many systems also store sparse (keyword) representations next to dense vectors to support hybrid search, the subject of Part 3.

8. Keeping the Index Healthy

An index is not a one-time artefact. Documents are added, revised and withdrawn, embedding models improve, and chunking strategies evolve. Without discipline, the index slowly fills with stale, duplicated and orphaned content, and users lose trust.

8.1 Incremental updates

- **Content hashing.** Store a hash of each source document and chunk. On re-ingestion, process only changed documents and delete the chunks of documents that vanished.
- **Stable identifiers.** Derive chunk IDs deterministically (for example, document ID plus chunk position plus hash) so that re-runs update rather than duplicate.
- **Soft deletes and versioning.** Mark superseded documents instead of erasing them, so that you can roll back and audit. Filter on a "current" flag at query time.
- **Embedding version.** Record which model and which chunking configuration produced each vector. Never mix vectors from different models in one index.
- **Blue-green re-indexing.** When changing the embedding model or chunker, build the new index alongside the old, evaluate it, and then switch traffic.

8.2 Deduplication and quality filters

Duplicate and near-duplicate chunks waste the top-k slots: four copies of the same paragraph leave room for only one other idea. Remove exact duplicates by hash and near-duplicates with MinHash or embedding similarity thresholds. Also filter out very short chunks, chunks that are mostly punctuation or navigation text, and chunks with low language-detection confidence.

8.3 Security and access control at index time

Attach access-control metadata (owner, group, classification) at ingestion and enforce it as a **mandatory filter** at query time, outside the control of the language model. Never rely on instructing the model to "not reveal" restricted content: once a restricted chunk is in the prompt, it can leak. Part 5 expands on security.

8.4 An indexing checklist

1. Parse a sample of real documents and read the output; fix reading order, tables and boilerplate first.
2. Choose a structure-aware splitter where structure exists; use recursive splitting otherwise.
3. Prepend heading paths or contextual blurbs to chunks before embedding.
4. Attach metadata for citations, filters, access control and updates.
5. Evaluate two or three chunk sizes and at least two embedding models on your own questions.
6. Adopt parent-document or sentence-window retrieval if answers need more context than precise matches.

7. Tune ANN parameters against a recall target, with realistic filters.
8. Version embeddings and chunking configuration; plan for re-indexing.
9. Deduplicate, and monitor index size, ingestion failures and staleness.

9. End-to-End Example: Indexing a Product-Manual Corpus

To make the ideas concrete, consider a corpus of 400 PDF product manuals in two languages, updated quarterly. Users ask installation, troubleshooting and warranty questions, often naming a product model. The design below combines several techniques from this document, each chosen to address a specific property of the data.

- **Structure-aware splitting by section**, because manuals have numbered headings. The heading path is prepended to every chunk.
- **Parent-child retrieval**, because troubleshooting answers need the whole procedure while matching needs a precise step.
- **Metadata** for product model, manual version, language and section, because users name a model and the same text differs between models.
- **Content hashing**, because quarterly updates change only a fraction of the manuals.

9.1 The metadata schema

Field	Example	Used for
doc_id	manual-x200-v3	Grouping, deletion, citation
product_model	X200	Exact filter from the user's question
version	3.1	Prefer the latest; allow historical lookups
language	en	Language filter and routing
section_path	Troubleshooting > Power > No LED	Citation and heading context
page	42	Citation with page number
content_hash	sha256:9f2c...	Skip unchanged chunks on re-ingestion
acl	public	Mandatory access filter

Table 3. A metadata schema for the manual corpus.

9.2 The ingestion loop

The loop below is deliberately simple but captures the key behaviours: it computes a document hash, skips unchanged documents, deletes old chunks of changed documents, and writes new chunks with deterministic identifiers. Replace the in-memory dictionary of hashes with a table in your database.

```
import hashlib
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings        # or any embeddings class

def sha(text: str) -> str:
    return hashlib.sha256(text.encode("utf-8")).hexdigest()

vs = Chroma(collection_name="manuals", embedding_function=OpenAIEmbeddings(),
            persist_directory="./index")
seen = load_hashes()                                  # {doc_id: hash} from your DB

def ingest(doc_id, raw_text, meta):
    h = sha(raw_text)
    if seen.get(doc_id) == h:
        return "unchanged"
    vs.delete(where={"doc_id": doc_id})               # remove stale chunks
    sections = md_splitter.split_text(raw_text)       # structure-aware (Listing 2)
    chunks = splitter.split_documents(sections)       # then size-limit (Listing 1)
    ids = []
    for i, c in enumerate(chunks):
        c.metadata.update(meta, doc_id=doc_id, content_hash=sha(c.page_content))
        ids.append(f"{doc_id}:{i}:{c.metadata['content_hash'][:8]}")
    vs.add_documents(chunks, ids=ids)
    seen[doc_id] = h
    return f"indexed {len(chunks)} chunks"
```

Listing 7. Idempotent ingestion with hashing, deletion of stale chunks and deterministic IDs.

9.3 Querying with metadata filters

At query time the application extracts the product model from the question (using the self-query technique from Part 1, or a regular expression for model codes) and applies it as a filter together with the mandatory access filter. The vector search then ranks only the relevant manual's chunks.

```
hits = vs.similarity_search(
    "LED does not turn on after plugging in",
    k=8,
    filter={"$and": [{"product_model": "X200"}, {"language": "en"}, {"acl": "public"}]},
)
for h in hits:
    print(h.metadata["section_path"], "| page", h.metadata["page"])
```

Listing 8. Filtered search; the access filter is applied by the application, never by the model.

10. Designing Indexing Experiments

Because indexing choices interact, resist the temptation to change several at once. Build a small, repeatable experiment harness so that every change produces a number. The evaluation set can start with 50 to 100 questions, each labelled with the chunk or section that contains the answer (a few hours of expert time, or an LLM-generated set reviewed by a human; see Part 5).

10.1 An experiment grid

Experiment	Variants to compare	Primary metric	Secondary metric
Chunk size	250, 500, 1000 tokens	Recall@5 of the answer-bearing chunk	Prompt tokens per query
Splitter	Recursive vs structure-aware	Recall@5, MRR	Chunk-size variance
Context enrichment	None, heading path, LLM blurb	Recall@5 on ambiguous questions	Ingestion cost
Retrieval unit	Flat vs parent-child	Answer correctness (end to end)	Latency
Embedding model	Three candidates	Recall@10, nDCG@10	Dimension, cost, latency
ANN setting	efSearch 32, 64, 128	Recall versus exact search	p95 latency

Table 4. A set of single-variable indexing experiments.

10.2 Troubleshooting guide

When retrieval quality disappoints, inspect the top results for a handful of failing questions before changing anything. The symptom usually points at the stage to fix.

Symptom	Likely cause	First fix to try
Right document found, wrong passage	Chunks too large or unfocused	Smaller child chunks with parent retrieval
Answer is split across two chunks	Boundary cuts an idea	More overlap; structure-aware splitting
Retrieved chunk is correct but model misreads it	Chunk lacks context (pronouns, table headers)	Heading path, contextual blurb, keep table headers
Exact codes and names not found	Dense-only search	Add keyword search (Part 3)
Top-k filled with near-identical text	Duplicates and boilerplate	Deduplicate; strip headers and footers
Good recall in tests, poor with filters	ANN filtering behaviour	Use filtered traversal; raise efSearch
Quality dropped after re-ingestion	Mixed embedding versions or parser change	Rebuild the index with one versioned configuration
Domain terms retrieved poorly	Embedding model mismatch	Try a domain model; fine-tune with hard negatives

Table 5. Symptom-driven troubleshooting for indexing issues.

> **TIP** Keep a persistent "failing questions" file. Each time a user reports a bad answer, add the question, the expected source passage and the diagnosis. Over months this becomes your most valuable regression test set.

10.3 Summary and further reading

A strong index separates the unit you search from the unit you read, carries context with every chunk, uses an embedding model matched to the domain, and is maintained as living infrastructure. The techniques here (structure-aware chunking, parent-child retrieval, multi-vector indexing, contextual enrichment, late chunking, Matryoshka embeddings, tuned HNSW) each address a specific failure and should be adopted in response to measured problems rather than all at once.

Key references: Chen et al., *Dense X Retrieval: What Retrieval Granularity Should We Use?* (2023); Sarthi et al., *RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval* (2024); Kusupati et al., *Matryoshka Representation Learning* (2022); Malkov and Yashunin, *Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs* (2016); Faysse et al., *ColPali: Efficient Document Retrieval with Vision Language Models* (2024); Anthropic, *Introducing Contextual Retrieval* (2024); Jina AI, *Late Chunking* (2024).

Next in the series: Part 3, *Hybrid Retrieval, Reranking and Context Assembly*, which covers how to search the index well.